

User Documentation for Algorithmic Trading System

Document Control

Document Title: User Documentation for Algorithmic Trading System

Version: 1.0

Date: April 24, 2025

Author: Senior Technical Writer, Hedge Fund

Status: Draft

Distribution: Trading Team, Product Owner, Compliance Team, Support Team, Stakeholders

Revision History

Version	Date	Author	Description
1.0	04/24/2025	Senior Technical Writer	Initial draft of User Documentation

Approval

Role	Name	Signature	Date
Product Owner	[To be assigned]		
Head of Trading	[To be assigned]		
Compliance Officer	[To be assigned]		
Support Lead	[To be assigned]		

Table of Contents

- 1. [Introduction](#)
 - 1.1. Purpose of the Document
 - 1.2. Scope of the Algorithmic Trading System
 - 1.3. Intended Audience
 - 1.4. Assumptions and Prerequisites
 - 1.5. Definitions and Acronyms
 - 1.6. How to Use This Document
- 2. [System Overview](#)
 - 2.1. System Architecture
 - 2.2. Key Features
 - 2.3. Supported Asset Classes and Exchanges
 - 2.4. Integration with Interactive Brokers
 - 2.5. Hardware and Software Requirements
 - 2.6. Limitations and Constraints
- 3. [Getting Started](#)
 - 3.1. System Installation
 - 3.2. Initial Configuration
 - 3.3. User Account Setup
 - 3.4. Accessing the System
 - 3.5. Navigating the Dashboard
- 4. [Core Functionality](#)
 - 4.1. Market Data Management
 - 4.1.1. Real-Time Data Feeds
 - 4.1.2. Historical Data Access
 - 4.1.3. Data Visualization
 - 4.2. Trading Strategies
 - 4.2.1. Trend-Following Strategy
 - 4.2.2. Support and Resistance Strategy
 - 4.2.3. Custom Strategy Configuration
 - 4.3. Order Execution
 - 4.3.1. Placing Market Orders
 - 4.3.2. Placing Limit Orders
 - 4.3.3. Placing Bracket Orders
 - 4.3.4. Order Monitoring and Cancellation

- 4.4. Risk Management
 - 4.4.1. Position Sizing
 - 4.4.2. Stop-Loss and Take-Profit
 - 4.4.3. Circuit Breaker Mechanism
 - 4.4.4. Portfolio Risk Limits
- 4.5. AI Agents Framework
 - 4.5.1. AI-Driven Trade Recommendations
 - 4.5.2. Risk Adjustment by AI
 - 4.5.3. Human Override Options
- 4.6. Backtesting
 - 4.6.1. Setting Up a Backtest
 - 4.6.2. Analyzing Backtest Results
 - 4.6.3. Exporting Backtest Reports
- 4.7. Reporting and Analytics
 - 4.7.1. Trade Performance Reports
 - 4.7.2. Portfolio Analytics
 - 4.7.3. Compliance Reports
 - 4.7.4. Exporting Reports
- 4.8. Monitoring and Alerts
 - 4.8.1. System Health Monitoring
 - 4.8.2. Performance Metrics
 - 4.8.3. Configuring Alerts
- 5. [User Interface Guide](#)
 - 5.1. Dashboard Overview
 - 5.2. Market Watch Panel
 - 5.3. Charting Tools
 - 5.4. Order Management Panel
 - 5.5. Portfolio View
 - 5.6. Strategy Configuration Panel
 - 5.7. Backtesting Interface
 - 5.8. Reporting Interface
 - 5.9. Settings and Customization
- 6. [Advanced Features](#)
 - 6.1. Custom Indicator Development
 - 6.2. API Access for Automation
 - 6.3. Multi-User Collaboration
 - 6.4. Localization and Regional Settings
 - 6.5. Integration with Third-Party Tools
- 7. [Security and Compliance](#)
 - 7.1. User Authentication and Authorization
 - 7.2. Data Encryption
 - 7.3. Audit Trails and Logging
 - 7.4. Regulatory Compliance (SEC, GDPR, MiFID II, SEBI)
 - 7.5. Data Retention Policies
- 8. [Troubleshooting and Support](#)
 - 8.1. Common Issues and Resolutions
 - 8.2. Error Messages and Their Meanings
 - 8.3. System Logs and Diagnostics
 - 8.4. Contacting Support
 - 8.5. Submitting Feedback
- 9. [Best Practices and Tips](#)
 - 9.1. Optimizing Trading Performance
 - 9.2. Managing Risk Effectively
 - 9.3. Backtesting Best Practices
 - 9.4. Dashboard Customization Tips
 - 9.5. Staying Compliant

10. [Missing Information and Assumptions](#)
 - 10.1. Missing Information
 - 10.2. Assumptions and Best Practices
 11. [Glossary](#)
 12. [Appendices](#)
 - 12.1. Sample Configuration Files
 - 12.2. Example OHLCV Data
 - 12.3. Example Trade Logs
 - 12.4. Keyboard Shortcuts
 - 12.5. References and Resources
-

1. Introduction

1.1. Purpose of the Document

This User Documentation provides a comprehensive guide for operating the Algorithmic Trading System (ATS), a sophisticated platform designed for paper trading across multiple asset classes. The document aims to equip users with the knowledge and skills needed to install, configure, and utilize the ATS effectively. It covers system setup, core functionalities, user interface navigation, advanced features, security protocols, troubleshooting, and best practices. By following this guide, users can maximize the system's capabilities while ensuring compliance with regulatory standards.

1.2. Scope of the Algorithmic Trading System

The ATS is a high-performance platform for algorithmic paper trading, integrated with the Interactive Brokers (IBKR) Paper Trading API. It supports trading in stocks, indices, forex, and commodities across global exchanges, including NYSE, NASDAQ, LSE, and NSE. Key features include:

- Real-time and historical market data processing using Polars and DuckDB.
- Automated trading strategies (Trend-Following, Support and Resistance).
- AI-driven trade recommendations and risk management.
- Order execution with market, limit, and bracket orders.
- Risk controls, including position sizing, stop-loss, and circuit breakers.
- Responsive web-based dashboard with charting and analytics.
- Backtesting for strategy validation.
- Comprehensive reporting and compliance logging.
- Monitoring and alerting for system performance.

The system is designed for local deployment on a Lenovo Legion Pro 5, targeting traders, portfolio managers, and compliance officers in a hedge fund environment. It does not support live trading or integration with brokers other than IBKR.

1.3. Intended Audience

This document is intended for:

- **Traders:** To execute trades, configure strategies, and analyze performance.
- **Portfolio Managers:** To monitor portfolios, manage risk, and generate reports.
- **Compliance Officers:** To ensure regulatory adherence and audit trails.
- **IT Administrators:** To install, configure, and maintain the system.
- **Support Staff:** To assist users and resolve issues.

Users are expected to have basic knowledge of trading concepts, web-based applications, and Python-based systems. Advanced features may require familiarity with APIs and custom indicator development.

1.4. Assumptions and Prerequisites

Assumptions:

- Users have access to a Lenovo Legion Pro 5 with Ubuntu 22.04.
- IBKR Paper Trading API credentials are available.
- Users are familiar with trading terminologies (e.g., OHLCV, stop-loss).
- The system is deployed locally, not on cloud infrastructure.
- Compliance requirements align with SEC, GDPR, MiFID II, and SEBI.

Prerequisites:

- Hardware: Lenovo Legion Pro 5 (NVIDIA RTX 3060, 32GB RAM).
- Software: Ubuntu 22.04, Docker, Python 3.10, Node.js, web browser (Chrome/Firefox).
- Network: Stable internet for IBKR API connectivity.

- Skills: Basic Linux commands, web navigation, and trading knowledge.

1.5. Definitions and Acronyms

- **ATS:** Algorithmic Trading System.
- **IBKR:** Interactive Brokers.
- **OHLCV:** Open, High, Low, Close, Volume.
- **VW SMA:** Volume-Weighted Simple Moving Average.
- **MFI:** Money Flow Index.
- **ATR:** Average True Range.
- **Polars:** High-performance data processing library.
- **DuckDB:** In-memory analytical database.
- **FastAPI:** Python web framework for APIs.
- **React:** JavaScript library for the dashboard.

1.6. How to Use This Document

This document is structured to guide users from initial setup to advanced operations. Beginners should start with Sections 3 (Getting Started) and 4 (Core Functionality). Experienced users can refer to Section 6 (Advanced Features) for customizations and APIs. Compliance officers should focus on Section 7 (Security and Compliance). Each section includes step-by-step instructions, screenshots, and examples. Use the Table of Contents to navigate, and refer to the Glossary and Appendices for additional resources.

2. System Overview

2.1. System Architecture

The ATS is a modular system comprising several services orchestrated via Docker Compose. The architecture includes:

- **Data Ingestion Service:** Fetches real-time and historical OHLCV data from IBKR API, caching in Redis.
- **Data Processing Service:** Normalizes data, calculates indicators (VW SMA, MFI, ATR) using Polars.
- **Data Storage Service:** Stores processed data, trade logs, and portfolio in DuckDB with Parquet partitioning.
- **Strategy Engine:** Generates trading signals based on Trend-Following and Support/Resistance strategies.
- **Order Execution Module:** Places and monitors orders via IBKR API.
- **Risk Management Module:** Enforces position sizing, stop-loss, and circuit breakers.
- **AI Agents Framework:** Provides AI-driven trade recommendations and risk adjustments.
- **Market Watch Dashboard:** Web-based UI built with React and Tailwind CSS, displaying charts and controls.
- **Reporting Module:** Generates trade, portfolio, and compliance reports.
- **Monitoring and Logging Service:** Tracks performance with Prometheus and Grafana, logs to DuckDB.

Services communicate via RabbitMQ for messaging and FastAPI for APIs. The system is deployed locally on a Lenovo Legion Pro 5.

2.2. Key Features

- **Real-Time Data:** Streams OHLCV data for 100 scrips across global exchanges.
- **Automated Strategies:** Pre-built Trend-Following and Support/Resistance strategies, customizable via parameters.
- **Order Types:** Market, limit, and bracket orders with stop-loss and take-profit.
- **Risk Management:** Dynamic position sizing, portfolio risk limits, and circuit breakers.
- **AI Integration:** AI agents optimize trade timing and risk using LSTM models and SHAP explanations.
- **Backtesting:** Simulates strategies on historical data (up to 1TB).
- **Dashboard:** Responsive UI with candlestick charts, watchlists, and order controls.
- **Reporting:** Trade performance, portfolio analytics, and compliance reports in CSV/PDF.
- **Monitoring:** Real-time latency and throughput metrics, configurable alerts.
- **Compliance:** Immutable audit trails, 7-year log retention, GDPR/MiFID II compliance.

2.3. Supported Asset Classes and Exchanges

- **Asset Classes:**
 - Stocks (e.g., AAPL, TSLA).
 - Indices (e.g., S&P 500, NIFTY 50).
 - Forex (e.g., EUR/USD, GBP/JPY).
 - Commodities (e.g., Gold, Crude Oil).
- **Exchanges:**
 - NYSE (New York Stock Exchange).
 - NASDAQ.
 - LSE (London Stock Exchange).
 - NSE (National Stock Exchange of India).

2.4. Integration with Interactive Brokers

The ATS integrates with the IBKR Paper Trading API for:

- Fetching real-time and historical OHLCV data (50 req/s limit).
- Placing and monitoring orders (100 msg/s limit).
- Simulating portfolio updates and trade executions.

Users must provide IBKR API credentials during configuration. The system caches data to handle API rate limits and downtime.

2.5. Hardware and Software Requirements

- **Hardware:**
 - Lenovo Legion Pro 5 (NVIDIA RTX 3060, 32GB RAM, 1TB SSD).
 - Minimum: 16GB RAM, 4-core CPU, 500GB SSD.
- **Software:**
 - OS: Ubuntu 22.04.
 - Docker: 20.10 or later.
 - Docker Compose: 1.29 or later.
 - Python: 3.10.
 - Node.js: 18.x.
 - Browser: Chrome 100+, Firefox 95+, Edge 100+.
- **Network:**
 - Stable internet (10 Mbps minimum).
 - Ports: 8000 (API), 3000 (dashboard), 5672 (RabbitMQ), 6379 (Redis).

2.6. Limitations and Constraints

- **Paper Trading Only:** No live trading support.
- **Single Broker:** Limited to IBKR Paper Trading API.
- **Local Deployment:** No cloud hosting, constrained by Lenovo Legion Pro 5.
- **API Limits:** 50 req/s for data, 100 msg/s for orders.
- **Scalability:** Supports up to 10 concurrent users, 1TB historical data.
- **Asset Classes:** Excludes cryptocurrencies and derivatives.
- **Mobile Support:** Web-based dashboard, no dedicated mobile app.

3. Getting Started

3.1. System Installation

To install the ATS on Ubuntu 22.04:

1. **Install Dependencies:**
2. `sudo apt update`
3. `sudo apt install -y docker.io docker-compose python3.10 python3-pip nodejs npm`
4. `sudo systemctl enable docker`
5. `sudo systemctl start docker`
6. **Clone Repository:**
7. `git clone https://github.com/hedgefund/ats.git`
8. `cd ats`
9. **Install Python Dependencies:**
10. `pip install poetry`
11. `poetry install`
12. **Install Node.js Dependencies:**

13. cd frontend
14. npm install
15. npm run build
16. cd ..
17. **Start Services:**
18. docker-compose up -d
19. **Verify Installation:**
 - o API: <http://localhost:8000/docs> (FastAPI Swagger UI).
 - o Dashboard: <http://localhost:3000>.
 - o Prometheus: <http://localhost:9090>.
 - o Grafana: <http://localhost:3001> (default login: admin/admin).

3.2. Initial Configuration

1. **Create .env File:**
2. cp .env.example .env

Edit .env with:

```
IBKR_API_KEY=your_api_key
IBKR_API_SECRET=your_api_secret
REDIS_HOST=redis
RABBITMQ_HOST=rabbitmq
DUCKDB_PATH=/data/ats.db
API_PORT=8000
DASHBOARD_PORT=3000
```

3. **Configure Keycloak:**
 - o Access Keycloak at <http://localhost:8080>.
 - o Create a realm named ats.
 - o Add users and roles (trader, admin, compliance).
 - o Enable MFA for traders.
4. **Set Up DuckDB:**
5. docker exec -it duckdb duckdb /data/ats.db
6. CREATE TABLE ohlcv (ticker VARCHAR, timestamp TIMESTAMP, open DOUBLE, high DOUBLE, low DOUBLE, close DOUBLE, volume BIGINT);
7. CREATE TABLE trade_log (trade_id VARCHAR, timestamp TIMESTAMP, ticker VARCHAR, strategy VARCHAR, action VARCHAR, quantity BIGINT, price DOUBLE, status VARCHAR);
8. CREATE TABLE portfolio (account_id VARCHAR, ticker VARCHAR, quantity BIGINT, avg_price DOUBLE);
9. **Restart Services:**
10. docker-compose restart

3.3. User Account Setup

1. **Log In to Keycloak:**
 - o URL: <http://localhost:8080>.
 - o Use admin credentials to create trader accounts.
 - o Example: Username: trader1, Password: P@ssw0rd123, Role: trader.
2. **Enable MFA:**
 - o In Keycloak, configure OTP for trader1.
 - o Trader scans QR code using Google Authenticator.
3. **Assign Permissions:**
 - o Traders: Access to dashboard, trading, and reporting.
 - o Admins: Full system access.
 - o Compliance: Access to audit logs and reports.

3.4. Accessing the System

1. **Open Dashboard:**
 - o URL: <http://localhost:3000>.
 - o Log in with Keycloak credentials.
 - o Enter MFA code if prompted.
2. **API Access:**
 - o URL: <http://localhost:8000/docs>.

- Obtain JWT token from Keycloak.
 - Use token in API requests (e.g., Authorization: Bearer <token>).
3. **Monitoring Tools:**
- Prometheus: <http://localhost:9090>.
 - Grafana: <http://localhost:3001>.

3.5. Navigating the Dashboard

The dashboard is a React-based UI with the following panels:

- **Market Watch:** Displays real-time OHLCV data for watchlist scrips.
- **Charts:** Candlestick charts with VW SMA, MFI, and ATR.
- **Order Management:** Place, monitor, and cancel orders.
- **Portfolio:** View positions and P&L.
- **Strategy Configuration:** Adjust strategy parameters.
- **Backtesting:** Run and analyze backtests.
- **Reports:** Generate and export reports.
- **Settings:** Customize layout, language, and alerts.

Navigation Tips:

- Use the sidebar to switch panels.
- Click the hamburger menu to collapse/expand the sidebar.
- Drag widgets to rearrange the layout.
- Save layout via Settings > Save Layout.

4. Core Functionality

4.1. Market Data Management

4.1.1. Real-Time Data Feeds

The ATS streams OHLCV data for up to 100 scrips from IBKR API.

Steps:

1. Go to Market Watch panel.
2. Add scrips (e.g., AAPL, TSLA) to the watchlist:
 - Click “Add Scrip”.
 - Enter ticker and exchange (e.g., AAPL:NASDAQ).
 - Click “Save”.
3. View real-time OHLCV data in the table.
4. Data updates every 5-10 seconds.

Notes:

- Rate limit: 50 req/s; system caches data in Redis.
- If IBKR API is down, cached data is used.

4.1.2. Historical Data Access

Access historical OHLCV data for backtesting and analysis.

Steps:

1. Go to Backtesting or Reporting panel.
2. Select “Historical Data”.
3. Specify:
 - Ticker (e.g., AAPL).
 - Timeframe (1m, 5m, 1h, 1d).
 - Date range (e.g., 2020-01-01 to 2025-04-24).
4. Click “Fetch Data”.
5. Data is displayed in a table or exported to CSV.

Notes:

- Data is stored in DuckDB with Parquet partitioning.
- Maximum data size: 1TB.

4.1.3. Data Visualization

Visualize OHLCV data with candlestick charts.

Steps:

1. Go to Charts panel.
2. Select a scrip from the watchlist.

3. Choose timeframe (1m, 5m, 1h, 1d).
4. Enable indicators:
 - VW SMA (55-day, 34-day, 21-day).
 - MFI (14-day).
 - ATR (21-day).
5. Zoom/pan using mouse or touch gestures.
6. Save chart as PNG via “Export Chart”.

Features:

- Plotly.js-based interactive charts.
- Customizable colors and layouts.
- Real-time updates for live data.

4.2. Trading Strategies

The ATS supports two pre-built strategies and custom configurations.

4.2.1. Trend-Following Strategy

Description: Generates buy/sell signals based on MFI and VW SMA crossovers.

Parameters:

- MFI Period: 14 (default).
- VW SMA Periods: 55, 34, 21 (default).
- Buy Condition: $MFI(14) > VW\ SMA(34)$, $price > VW\ SMA(55)$.
- Sell Condition: $MFI(14) < VW\ SMA(21)$.

Steps:

1. Go to Strategy Configuration panel.
2. Select “Trend-Following”.
3. Adjust parameters or use defaults.
4. Select scrips (e.g., AAPL, TSLA).
5. Click “Activate Strategy”.
6. Monitor signals in the Order Management panel.

4.2.2. Support and Resistance Strategy

Description: Generates signals based on price breaking support (S1) or resistance (R1) levels.

Parameters:

- Pivot Period: 5 days (default).
- Volume Threshold: 2x average (default).
- Buy Condition: Price breaks R1 with high volume.
- Sell Condition: Price breaks S1.

Steps:

1. Go to Strategy Configuration panel.
2. Select “Support and Resistance”.
3. Adjust parameters or use defaults.
4. Select scrips.
5. Click “Activate Strategy”.
6. Monitor signals in the Order Management panel.

4.2.3. Custom Strategy Configuration

Steps:

1. Go to Strategy Configuration > Custom Strategy.
2. Define rules using a Python-like syntax:
3. `if mfi(14) > vw_sma(34) and close > vw_sma(55):`
4. `return "BUY"`
5. `elif mfi(14) < vw_sma(21):`
6. `return "SELL"`
7. `else:`
8. `return "HOLD"`
9. Test rules in the Backtesting panel.
10. Save and activate the strategy.

Notes:

- Custom strategies require validation to prevent crashes.

- Contact support for complex rule development.

4.3. Order Execution

4.3.1. Placing Market Orders

Steps:

1. Go to Order Management panel.
2. Click “New Order”.
3. Select:
 - Scrip (e.g., AAPL).
 - Action: Buy/Sell.
 - Order Type: Market.
 - Quantity (e.g., 100 shares).
4. Click “Submit”.
5. Confirm order in the dialog (requires MFA).

Notes:

- Orders are executed at the current market price.
- IBKR rate limit: 100 msg/s.

4.3.2. Placing Limit Orders

Steps:

1. Go to Order Management panel.
2. Click “New Order”.
3. Select:
 - Scrip (e.g., AAPL).
 - Action: Buy/Sell.
 - Order Type: Limit.
 - Quantity (e.g., 100 shares).
 - Limit Price (e.g., 150.90).
4. Click “Submit”.
5. Confirm order (requires MFA).

Notes:

- Orders execute only at or better than the limit price.
- Monitor status in the Order Management panel.

4.3.3. Placing Bracket Orders

Steps:

1. Go to Order Management panel.
2. Click “New Order”.
3. Select:
 - Scrip (e.g., AAPL).
 - Action: Buy/Sell.
 - Order Type: Bracket.
 - Quantity (e.g., 100 shares).
 - Stop-Loss: $2 \times \text{ATR}$ (default).
 - Take-Profit: $3 \times \text{ATR}$ (default).
4. Click “Submit”.
5. Confirm order (requires MFA).

Notes:

- Bracket orders include a parent order, stop-loss, and take-profit.
- ATR is calculated over 21 periods.

4.3.4. Order Monitoring and Cancellation

Steps:

1. Go to Order Management panel.
2. View open orders in the “Open Orders” tab.
3. To cancel:
 - Select order.
 - Click “Cancel”.
 - Confirm cancellation (requires MFA).
4. View order history in the “Order History” tab.

Notes:

- Status updates (PENDING, FILLED, CANCELED) are displayed in real-time.
- Logs are stored in DuckDB for auditing.

4.4. Risk Management**4.4.1. Position Sizing**

Description: Calculates trade quantity based on account balance and risk percentage.

Formula:

$\text{Quantity} = (\text{Risk \%} \times \text{Account Balance}) / (\text{ATR} \times \text{Multiplier})$

- Default: Risk = 1%, Multiplier = 2.

Steps:

1. Go to Settings > Risk Management.
2. Set:
 - Risk Percentage (e.g., 1%).
 - ATR Multiplier (e.g., 2).
3. Save settings.
4. System automatically sizes positions for new orders.

4.4.2. Stop-Loss and Take-Profit**Steps:**

1. When placing a bracket order, specify:
 - Stop-Loss: $2 \times \text{ATR}$ (default).
 - Take-Profit: $3 \times \text{ATR}$ (default).
2. Alternatively, set global defaults in Settings > Risk Management.
3. Monitor stop-loss/take-profit triggers in Order Management.

Notes:

- Stop-loss prevents excessive losses.
- Take-profit locks in profits.

4.4.3. Circuit Breaker Mechanism

Description: Halts trading if market volatility exceeds 5% in 5 minutes.

Steps:

1. Go to Settings > Risk Management.
2. Enable "Circuit Breaker".
3. Set threshold (default: 5% in 5 minutes).
4. If triggered, trading pauses, and an alert is sent via email/SMS.

Notes:

- Manual override requires admin approval.
- Alerts are logged in DuckDB.

4.4.4. Portfolio Risk Limits**Steps:**

1. Go to Settings > Risk Management.
2. Set:
 - Maximum Open Risk: 10% (default).
 - Maximum Drawdown: 20% (default).
3. If limits are breached, new trades are rejected.
4. Monitor risk metrics in the Portfolio panel.

Notes:

- Risk is calculated as a percentage of account balance.
- Breaches trigger alerts.

4.5. AI Agents Framework**4.5.1. AI-Driven Trade Recommendations**

Description: AI agents use LSTM models to optimize trade timing.

Steps:

1. Go to Strategy Configuration > AI Agents.
2. Enable "AI Recommendations".
3. Select scrips (e.g., AAPL).
4. View recommendations in the Order Management panel.
5. Approve or reject recommendations (requires MFA).

Notes:

- SHAP explanations are logged for transparency.
- Confidence threshold: 0.5 (default).

4.5.2. Risk Adjustment by AI

Description: AI adjusts position sizes based on portfolio exposure.

Steps:

1. Go to Settings > AI Agents.
2. Enable “AI Risk Adjustment”.
3. Set maximum adjustment (e.g., 50% of default size).
4. Monitor adjustments in the Portfolio panel.

Notes:

- Adjustments prioritize portfolio stability.
- Logs are stored for auditing.

4.5.3. Human Override Options

Steps:

1. Go to Order Management panel.
2. For AI recommendations, click “Override”.
3. Specify manual trade parameters.
4. Submit order (requires MFA).

Notes:

- Overrides are logged with reasons.
- AI resumes after manual intervention.

4.6. Backtesting

4.6.1. Setting Up a Backtest

Steps:

1. Go to Backtesting panel.
2. Select:
 - Strategy (Trend-Following, Support/Resistance, Custom).
 - Scrip (e.g., AAPL).
 - Timeframe (1m, 5m, 1h, 1d).
 - Date Range (e.g., 2020-01-01 to 2025-04-24).
 - Initial Capital (e.g., £10,000).
3. Click “Run Backtest”.
4. Wait for results (typically <60s for 1TB data).

Notes:

- Historical data is sourced from DuckDB.
- Missing data triggers warnings.

4.6.2. Analyzing Backtest Results

Metrics:

- Total Return (%).
- Annualized Sharpe Ratio.
- Maximum Drawdown (%).
- Win Rate (%).
- Profit Factor.

Steps:

1. View results in the Backtesting panel.
2. Analyze metrics in tables and charts.
3. Compare multiple runs using “Compare Backtests”.
4. Save results via “Save Backtest”.

4.6.3. Exporting Backtest Reports

Steps:

1. In Backtesting panel, click “Export Report”.
2. Select format (CSV, PDF).
3. Specify output path or download directly.
4. Verify report contents (metrics, trade logs).

Notes:

- Reports are stored in DuckDB for auditing.
- PDF reports include charts.

4.7. Reporting and Analytics**4.7.1. Trade Performance Reports****Steps:**

1. Go to Reporting panel.
2. Select “Trade Performance”.
3. Specify:
 - Account ID.
 - Date Range (e.g., 2025-01-01 to 2025-04-24).
4. Click “Generate Report”.
5. View metrics (P&L, Sharpe Ratio, Win Rate).

4.7.2. Portfolio Analytics**Steps:**

1. Go to Reporting panel.
2. Select “Portfolio Analytics”.
3. Specify Account ID.
4. View metrics:
 - Exposure (%).
 - Value at Risk (VaR).
 - Sector Allocation.
5. Save or export report.

4.7.3. Compliance Reports**Steps:**

1. Go to Reporting panel.
2. Select “Compliance Report”.
3. Specify:
 - Account ID.
 - Date Range.
4. Generate report with:
 - Trade logs.
 - User actions.
 - Audit trails.
5. Export to CSV/PDF for regulatory submission.

4.7.4. Exporting Reports**Steps:**

1. In Reporting panel, click “Export”.
2. Select format (CSV, PDF).
3. Download or save to a specified path.
4. Verify report integrity.

Notes:

- Reports are immutable for compliance.
- Maximum report size: 1M trades.

4.8. Monitoring and Alerts**4.8.1. System Health Monitoring****Steps:**

1. Go to Monitoring panel.
2. View metrics:
 - API Latency (<100ms target).
 - Throughput (10,000 updates/s target).
 - Uptime (99.9% target).
3. Access Grafana for detailed dashboards:
 - URL: <http://localhost:3001>.
 - Login: admin/admin.

4.8.2. Performance Metrics

Metrics:

- Data Ingestion Latency.
- Order Execution Time.
- Backtest Execution Time.
- Dashboard Refresh Rate.

Steps:

1. In Monitoring panel, select “Performance Metrics”.
2. Filter by service (e.g., Data Ingestion).
3. View trends over time.
4. Export metrics to CSV.

4.8.3. Configuring Alerts**Steps:**

1. Go to Settings > Alerts.
2. Configure:
 - Latency Threshold: >100ms.
 - Error Rate: >1%.
 - Circuit Breaker: >5% price change.
3. Set notification channels (email, SMS).
4. Test alerts via “Send Test Alert”.

Notes:

- Alerts are logged in DuckDB.
 - Maximum alert frequency: 1/min.
-

5. User Interface Guide**5.1. Dashboard Overview**

The dashboard is a single-page React application with a responsive layout. Key components:

- **Sidebar:** Navigation to panels (Market Watch, Charts, etc.).
- **Header:** User profile, settings, and logout.
- **Main Panel:** Displays selected panel content.
- **Footer:** System status and version.

Access:

- URL: <http://localhost:3000>.
- Requires Keycloak login and MFA.

5.2. Market Watch Panel**Features:**

- Real-time OHLCV table.
- Watchlist management.
- Quick order placement.

Steps:

1. Click “Market Watch” in the sidebar.
2. Add scrips to watchlist:
 - Click “Add Scrip”.
 - Enter ticker (e.g., AAPL).
 - Select exchange (e.g., NASDAQ).
3. View OHLCV data (updates every 5-10s).
4. Click a scrip to view charts or place orders.

5.3. Charting Tools**Features:**

- Candlestick charts with Plotly.js.
- Indicators (VW SMA, MFI, ATR).
- Zoom, pan, and export options.

Steps:

1. Click “Charts” in the sidebar.
2. Select a scrip and timeframe.
3. Enable indicators via the toolbar.

4. Use mouse to zoom/pan.
5. Export chart via “Save as PNG”.

5.4. Order Management Panel

Features:

- Place, monitor, and cancel orders.
- View open orders and history.
- AI recommendations.

Steps:

1. Click “Order Management” in the sidebar.
2. Place a new order (see Section 4.3).
3. Monitor open orders in the “Open Orders” tab.
4. Cancel orders via “Cancel” button.
5. View history in the “Order History” tab.

5.5. Portfolio View

Features:

- Display positions, P&L, and exposure.
- Risk metrics (VaR, drawdown).
- Export portfolio data.

Steps:

1. Click “Portfolio” in the sidebar.
2. View:
 - Positions (ticker, quantity, avg price).
 - P&L (daily, total).
 - Risk Metrics.
3. Export to CSV via “Export Portfolio”.

5.6. Strategy Configuration Panel

Features:

- Configure pre-built and custom strategies.
- Activate/deactivate strategies.
- Test strategies via backtesting.

Steps:

1. Click “Strategy Configuration” in the sidebar.
2. Select strategy (Trend-Following, Custom).
3. Adjust parameters (e.g., MFI period).
4. Activate strategy via “Activate”.
5. Test via “Run Backtest”.

5.7. Backtesting Interface

Features:

- Run backtests on historical data.
- Analyze results (returns, Sharpe).
- Compare multiple backtests.

Steps:

1. Click “Backtesting” in the sidebar.
2. Set up backtest (see Section 4.6.1).
3. View results in tables/charts.
4. Compare runs via “Compare Backtests”.
5. Export results via “Export Report”.

5.8. Reporting Interface

Features:

- Generate trade, portfolio, and compliance reports.
- Export to CSV/PDF.
- Schedule recurring reports.

Steps:

1. Click “Reporting” in the sidebar.
2. Select report type (e.g., Trade Performance).

3. Specify parameters (e.g., date range).
4. Generate and view report.
5. Export via “Export”.

5.9. Settings and Customization

Features:

- Customize dashboard layout.
- Set language and regional formats.
- Configure alerts and risk parameters.

Steps:

1. Click “Settings” in the sidebar.
2. Adjust:
 - Layout: Drag widgets, save via “Save Layout”.
 - Language: English, Hindi, French.
 - Alerts: Latency, error thresholds.
 - Risk: Position sizing, circuit breaker.
3. Save changes.

6. Advanced Features

6.1. Custom Indicator Development

Steps:

1. Create a Python module in `src/indicators/`:
2. `def custom_indicator(data, period=14):`
3. `# Example: RSI`
4. `delta = data['close'].diff()`
5. `gain = (delta.where(delta > 0, 0)).rolling(window=period).mean()`
6. `loss = (-delta.where(delta < 0, 0)).rolling(window=period).mean()`
7. `rs = gain / loss`
8. `return 100 - (100 / (1 + rs))`
9. Register in `src/config/indicators.py`:
10. `INDICATORS['rsi'] = custom_indicator`
11. Restart services:
12. `docker-compose restart`
13. Use indicator in Strategy Configuration or Charts.

Notes:

- Test indicators in backtesting first.
- Contact support for complex indicators.

6.2. API Access for Automation

Endpoints:

- GET /market-data: Fetch OHLCV.
- POST /orders: Place orders.
- GET /portfolio: View portfolio.
- POST /backtest: Run backtests.

Steps:

1. Obtain JWT token from Keycloak.
2. Use Postman or Python requests:
3. `import requests`
4. `headers = {'Authorization': 'Bearer <token>'}`
5. `response = requests.get('http://localhost:8000/market-data?ticker=AAPL', headers=headers)`
6. `print(response.json())`
7. Refer to <http://localhost:8000/docs> for API documentation.

Notes:

- Rate limit: 100 req/s.
- Requires MFA for order endpoints.

6.3. Multi-User Collaboration

Steps:

1. Assign roles in Keycloak (trader, admin, compliance).

2. Share watchlists:
 - In Market Watch, click “Share Watchlist”.
 - Select users or roles.
 - Save.
3. Collaborate on strategies:
 - In Strategy Configuration, click “Share Strategy”.
 - Select users.
 - Save.
4. View shared resources in respective panels.

Notes:

- Maximum 10 concurrent users.
- Audit logs track shared resource access.

6.4. Localization and Regional Settings

Steps:

1. Go to Settings > Localization.
2. Select:
 - Language: English, Hindi, French.
 - Date Format: MM/DD/YYYY, DD/MM/YYYY.
 - Currency: USD, GBP, INR.
3. Save changes.
4. Dashboard updates to reflect settings.

Notes:

- Fallback language: English.
- Regional formats apply to reports and charts.

6.5. Integration with Third-Party Tools

Supported Tools:

- Prometheus: Metrics collection.
- Grafana: Visualization.
- RabbitMQ: Messaging.
- Redis: Caching.

Steps:

1. Configure integrations in .env:
2. PROMETHEUS_HOST=prometheus
3. GRAFANA_HOST=grafana
4. RABBITMQ_HOST=rabbitmq
5. REDIS_HOST=redis
6. Access tools:
 - Prometheus: <http://localhost:9090>.
 - Grafana: <http://localhost:3001>.
7. Monitor metrics or queues as needed.

Notes:

- Ensure tools are running via Docker Compose.
- Contact support for custom integrations.

7. Security and Compliance

7.1. User Authentication and Authorization

- **Authentication:** Keycloak with JWT tokens and MFA.
- **Authorization:** Role-based access (trader, admin, compliance).
- **Steps:**
 1. Log in at <http://localhost:3000>.
 2. Enter Keycloak credentials.
 3. Provide MFA code via Google Authenticator.
 4. Access granted based on role.

Notes:

- Tokens expire after 30 minutes.

- MFA is mandatory for traders.

7.2. Data Encryption

- **API:** TLS 1.3 for all communications.
- **Storage:** AES-256 for DuckDB Parquet files.
- **Credentials:** Stored in HashiCorp Vault.
- **Steps:**
 1. Verify TLS in browser (lock icon).
 2. Ensure .env includes Vault configuration:
 3. VAULT_HOST=vault
 4. VAULT_TOKEN=your_token

7.3. Audit Trails and Logging

- **Logs:** Stored in DuckDB, immutable.
- **Contents:** Trade actions, user logins, API calls, errors.
- **Steps:**
 1. Go to Reporting > Compliance Report.
 2. Query logs by user, date, or action.
 3. Export for auditing.

Notes:

- Logs are retained for 7 years.
- Checksum validation ensures integrity.

7.4. Regulatory Compliance

- **Standards:** SEC, GDPR, MiFID II, SEBI.
- **Features:**
 - Immutable trade logs.
 - GDPR-compliant data anonymization.
 - MiFID II transaction reporting.
 - SEBI market abuse detection.
- **Steps:**
 1. Generate compliance reports (Section 4.7.3).
 2. Submit to regulators as needed.
 3. Respond to GDPR data requests via admin panel.

7.5. Data Retention Policies

- **Policy:** Retain trade logs and user actions for 7 years.
- **Steps:**
 1. Verify retention in Reporting > Compliance Report.
 2. Schedule backups in Settings > Data Management.
 3. Delete expired user data via admin panel.

Notes:

- Backups are encrypted with AES-256.
- Contact compliance officer for retention queries.

8. Troubleshooting and Support

8.1. Common Issues and Resolutions

- **Issue:** Dashboard not loading.
 - **Resolution:** Check <http://localhost:3000>, restart Docker Compose (docker-compose restart).
- **Issue:** IBKR API errors.
 - **Resolution:** Verify API key in .env, check IBKR status.
- **Issue:** Slow performance.
 - **Resolution:** Monitor Grafana, optimize DuckDB queries.
- **Issue:** MFA failure.
 - **Resolution:** Resync Google Authenticator, contact admin.

8.2. Error Messages and Their Meanings

- **invalid_ticker:** Ticker not found in IBKR.
- **rate_limit_exceeded:** Exceeded IBKR API limits; wait 1s.

- **risk_violation:** Trade exceeds risk limits.
- **schema_mismatch:** Invalid data format; check input.

8.3. System Logs and Diagnostics

Steps:

1. Go to Monitoring > Logs.
2. Filter by level (INFO, ERROR, WARNING).
3. Export logs to CSV.
4. Access raw logs:
5. docker logs ats_api

Notes:

- Logs are stored in DuckDB.
- Maximum log size: 1TB.

8.4. Contacting Support

- **Email:** support@hedgefund.com.
- **Phone:** +1-800-555-1234 (9 AM–5 PM EST).
- **Ticket System:** <http://localhost:8080/support>.
- **Steps:**
 1. Submit ticket with issue description, screenshots, and logs.
 2. Expect response within 24 hours.
 3. Escalate critical issues to Head of Trading.

8.5. Submitting Feedback

Steps:

1. Go to Settings > Feedback.
2. Enter feedback (e.g., feature request, bug report).
3. Attach screenshots if applicable.
4. Submit.

Notes:

- Feedback is reviewed weekly.
- Anonymized feedback may be shared with stakeholders.

9. Best Practices and Tips

9.1. Optimizing Trading Performance

- **Monitor Latency:** Ensure API latency <100ms via Grafana.
- **Cache Data:** Use Redis to reduce IBKR API calls.
- **Limit Scripts:** Restrict watchlist to 50 scrips for optimal performance.
- **Backtest First:** Validate strategies before activation.

9.2. Managing Risk Effectively

- **Set Conservative Limits:** Use 1% risk per trade.
- **Enable Circuit Breakers:** Protect against volatility.
- **Monitor Drawdown:** Pause trading if drawdown >20%.
- **Review AI Adjustments:** Verify AI risk changes daily.

9.3. Backtesting Best Practices

- **Use Long Periods:** Test over 5+ years for robustness.
- **Vary Conditions:** Test bull, bear, and sideways markets.
- **Validate Metrics:** Ensure Sharpe Ratio >1.0.
- **Compare Strategies:** Run multiple backtests to optimize.

9.4. Dashboard Customization Tips

- **Save Layouts:** Save multiple layouts for different tasks.
- **Use Shortcuts:** Enable keyboard shortcuts (Appendix 12.4).
- **High Contrast Mode:** Enable for accessibility.
- **Minimize Widgets:** Reduce clutter for faster rendering.

9.5. Staying Compliant

- **Generate Reports Regularly:** Monthly compliance reports.
- **Review Logs:** Weekly audit trail checks.
- **Train Users:** Ensure all traders understand GDPR/MiFID II.

- **Backup Data:** Schedule weekly backups.

10. Missing Information and Assumptions

10.1. Missing Information

- Exact IBKR API mock data specifications.
- Detailed trader personas for usability.
- Specific compliance requirements for paper trading.
- News feed integration details.
- Custom indicator library specifications.

10.2. Assumptions and Best Practices

- **Mock Data:** Synthetic OHLCV based on market patterns. **Reason:** Ensures realistic testing without live data.
- **Compliance:** 7-year log retention, immutable audit trails. **Reason:** Aligns with SEC/GDPR standards.
- **Usability:** Assumed intermediate traders. **Reason:** Matches hedge fund user base.
- **Performance:** Targeted <100ms latency. **Reason:** High-frequency trading standard.
- **Security:** TLS 1.3, AES-256 encryption. **Reason:** Industry-standard for secure systems.

11. Glossary

- **ATS:** Algorithmic Trading System.
- **IBKR:** Interactive Brokers.
- **OHLCV:** Open, High, Low, Close, Volume.
- **VW SMA:** Volume-Weighted Simple Moving Average.
- **MFI:** Money Flow Index.
- **ATR:** Average True Range.
- **Polars:** Data processing library.
- **DuckDB:** Analytical database.
- **FastAPI:** Web framework for APIs.
- **React:** JavaScript library for UI.

12. Appendices

12.1. Sample Configuration Files

.env:

```
IBKR_API_KEY=your_api_key
IBKR_API_SECRET=your_api_secret
REDIS_HOST=redis
RABBITMQ_HOST=rabbitmq
DUCKDB_PATH=/data/ats.db
API_PORT=8000
DASHBOARD_PORT=3000
VAULT_HOST=vault
VAULT_TOKEN=your_token
```

docker-compose.yml:

```
version: '3.8'
services:
  api:
    build: ./api
    ports:
      - "8000:8000"
    env_file: .env
  frontend:
    build: ./frontend
    ports:
      - "3000:3000"
  redis:
```

```
image: redis:latest
rabbitmq:
  image: rabbitmq:3-management
duckdb:
  image: duckdb:latest
volumes:
  - ./data:/data
prometheus:
  image: prom/prometheus
ports:
  - "9090:9090"
grafana:
  image: grafana/grafana
ports:
  - "3001:3000"
```

12.2. Example OHLCV Data

```
{
  "ticker": "AAPL",
  "exchange": "NASDAQ",
  "timestamp": "2025-01-01T10:00:00Z",
  "open": 150.25,
  "high": 151.10,
  "low": 149.80,
  "close": 150.90,
  "volume": 1000000
}
```

12.3. Example Trade Logs

```
{
  "trade_id": "T12345",
  "timestamp": "2025-01-01T10:00:00Z",
  "ticker": "AAPL",
  "strategy": "trend_following",
  "action": "BUY",
  "quantity": 100,
  "price": 150.90,
  "status": "FILLED"
}
```

12.4. Keyboard Shortcuts

- **Ctrl+Shift+M**: Open Market Watch.
- **Ctrl+Shift+C**: Open Charts.
- **Ctrl+Shift+O**: Open Order Management.
- **Ctrl+Shift+P**: Open Portfolio.
- **Ctrl+Shift+S**: Save Layout.

12.5. References and Resources

- IBKR API: <https://interactivebrokers.github.io/tws-api/>
- Polars: <https://pola.rs/>
- DuckDB: <https://duckdb.org/>
- FastAPI: <https://fastapi.tiangolo.com/>
- React: <https://reactjs.org/>
- WCAG 2.1: <https://www.w3.org/TR/WCAG21/>